

OPERATIONS RUNBOOK

Operations Runbook

dealing cluster

CLUSTER dealing cluster

AUDIENCE Platform & on-call engineers

CLASSIFICATION Internal — keep current as the platform evolves

Contents

01	Quick reference	3
02	How to access	4
03	Health-check one-liners (the "is anything wrong?" pass)	4
04	Incident runbooks	5
05	Common operations	13
06	Cheat sheet	16
07	Escalation matrix (template — TEAM TO FILL IN)	17
08	References	17

Companion to `workload-catalog-dealing.md`. The catalog says *what's deployed*; this runbook says *what to do when it breaks*.

Iterate this doc every time you handle an incident — add the symptoms you saw, the steps that worked, and what would have helped you find the answer faster.

Quick reference

Thing	Value
Cluster name	<code>dealing</code>
Control-plane VIP	<code>10.10.120.138:6443</code> (kube-vip leader election across masters)
Masters	<code>dealing-m-1</code> (10.10.120.131), <code>dealing-m-2</code> (.132), <code>dealing-m-3</code> (.133)
Workers	<code>dealing-w-1</code> (.134), <code>dealing-w-2</code> (.135), <code>dealing-w-3</code> (.136)
Ingress LB IP	<code>10.10.120.140</code> (Cilium IngressController, shared mode)
Kubeconfig	<code>clusters/dealing/kubeconfig.yaml</code> (in repo)
SSH user	<code>ubuntu</code> with key <code>~/.ssh/rke2_cluster_id</code>
RKE2 version	<code>v1.32.10+rke2r1</code>
CNI	Cilium 1.18.3 (kubeProxyReplacement, WireGuard pod-to-pod, L2 announcements)
Observability	central Mimir (<code>mimir.stackflow.org</code>), Loki (<code>loki.stackflow.org</code>), Grafana (<code>10.10.103.203:3000</code>)
Alerting	Grafana managed alerts → Microsoft Teams webhook
Etcd quota	8 GB (<code>quota-backend-bytes=8589934592</code>)
Etcd snapshots	every 6h, retained 10, stored on <code>scsi1</code> disk per master

How to access

```
# Cluster API (via VIP)
export KUBECONFIG=$(pwd)/clusters/dealing/kubeconfig.yaml
kubectl get nodes

# SSH to a node
ssh -i ~/.ssh/rke2_cluster_id ubuntu@10.10.120.131

# Cilium debugging – exec into the agent on a specific node
M1_AGENT=$(kubectl -n kube-system get pod -l k8s-app=cilium --field-selector spec.nodeName=dealing-m-1 -o jsonpath='{.items[0].metadata.name}')
kubectl -n kube-system exec -it $M1_AGENT -c cilium-agent -- cilium-dbg status --verbose
```

Health-check one-liners (the "is anything wrong?" pass)

Run these in order — anything red means stop and dig in before moving on.

```
# 1. All nodes Ready?
kubectl get nodes

# 2. All control-plane components healthy?
kubectl get --raw '/readyz?verbose' | grep -v ok$

# 3. Etcd quorum + DB size
kubectl -n kube-system get pods -l component=etcd -o wide
# (Detail per pod: kubectl exec -- etcdctl endpoint status --cluster)

# 4. Any pods in trouble?
kubectl get pods -A --field-selector=status.phase!=Running,status.phase!=Succeeded

# 5. Recent events (last 10 minutes)
kubectl get events -A --sort-by='.lastTimestamp' | tail -40

# 6. Cilium agents healthy on every node?
kubectl -n kube-system get pods -l k8s-app=cilium -o wide
# all should show 1/1 Running

# 7. Are Prometheus scrapes reaching their targets?
# (Visit Grafana → Connections → Data sources → Mimir → "Test"; or in Prometheus pod)
kubectl -n monitoring exec prometheus-kube-prometheus-stack-prometheus-0 -c prometheus -- \
  wget -qO- "http://localhost:9090/api/v1/query?query=count(up==0)"

# 8. Is anything dropping packets right now?
W1_AGENT=$(kubectl -n kube-system get pod -l k8s-app=cilium --field-selector spec.nodeName=dealing-w-1 -o jsonpath='{.items[0].metadata.name}')
kubectl -n kube-system exec $W1_AGENT -c cilium-agent -- hubble observe --verdict DROPPED --
last 50
```

Incident runbooks

IR-1 — Apiserver slow or unreachable

Symptoms

- `kubectl` commands time out or hang
- Grafana alert: `apiserver:request_p99 > 1s` or `up{job="apiserver"} == 0`
- Application pods can't talk to the API (watch reconnect loops)

Diagnosis

```
# 1. Can you reach any master directly?
for ip in 10.10.120.131 10.10.120.132 10.10.120.133; do
  echo "--- $ip ---"
  curl -sk --max-time 5 https://$ip:6443/livez 2>&1 | head -3
done

# 2. Is the VIP responding?
curl -sk --max-time 5 https://10.10.120.138:6443/livez

# 3. Which master holds the VIP right now?
for ip in 10.10.120.131 10.10.120.132 10.10.120.133; do
  ssh -i ~/.ssh/rke2_cluster_id ubuntu@$ip "ip addr show eth0 | grep 10.10.120.138" 2>/dev/
  null && echo " → VIP on $ip"
done

# 4. kube-apiserver logs on each master
ssh ubuntu@10.10.120.131 "sudo journalctl -u rke2-server --since '15 min ago' --no-pager |
tail -100"
```

Common causes

- etcd slow → apiserver slow (see IR-2)
- kube-vip leader election flap → VIP keeps moving (look at `kube-vip-ds` logs in kube-system)
- Master node out of memory / disk full
- Massive watch storm (count `etcd_server_watch_stream`)

Remediation

- If one master is bad → drain it, let other masters take over (`kubectl drain dealing-m-N --ignore-daemonsets --delete-emptydir-data`)
- If VIP is flapping → restart `kube-vip-ds` pods one by one (`kubectl -n kube-system delete pod -l name=kube-vip-ds`)
- If etcd-driven → see IR-2

Verification

- All three `https://<master-ip>:6443/livez` return `ok`
- `https://10.10.120.138:6443/livez` returns `ok`
- `kubectl get --raw '/readyz?verbose'` clean

Escalation: Page after 10 min if VIP unreachable. Cluster is effectively offline.

IR-2 — Etcd is degraded (slow fsync, growing DB, or no leader)

Symptoms

- Grafana alert: `etcd_disk_wal_fsync_duration_seconds:p99 > 100ms` or `etcd_server_has_leader == 0`
- Apiserver request latency climbing
- `kubectl` commands feel sluggish
- DB size approaching the 8 GB quota: `etcd_mvcc_db_total_size_in_bytes / 8589934592 > 0.85`

Diagnosis

```
# Health of every etcd member
M1=$(kubectl -n kube-system get pod -l component=etcd --field-selector spec.nodeName=dealing -m-1 -o name | head -1)
kubectl -n kube-system exec $M1 -c etcd -- /bin/sh -c '
ETCDCTL_API=3 etcdctl \
  --cacert=/var/lib/rancher/rke2/server/tls/etcd/server-ca.crt \
  --cert=/var/lib/rancher/rke2/server/tls/etcd/server-client.crt \
  --key=/var/lib/rancher/rke2/server/tls/etcd/server-client.key \
  --endpoints=https://10.10.120.131:2379,https://10.10.120.132:2379,https://10.10.120.133:2379 \
  endpoint health --cluster -w table'
# Repeat with `endpoint status --cluster -w table` for DB size, raft index, leader

# Top contributors to DB growth
ssh ubuntu@10.10.120.131 "sudo du -sh /var/lib/rancher/rke2/server/db/* 2>/dev/null"

# What's filling etcd? Check key counts by prefix
kubectl -n kube-system exec $M1 -c etcd -- /bin/sh -c '
ETCDCTL_API=3 etcdctl \
  --cacert=/var/lib/rancher/rke2/server/tls/etcd/server-ca.crt \
  --cert=/var/lib/rancher/rke2/server/tls/etcd/server-client.crt \
  --key=/var/lib/rancher/rke2/server/tls/etcd/server-client.key \
  get --prefix --keys-only / | cut -d/ -f1-3 | sort | uniq -c | sort -rn | head -20'
```

Note: the RKE2 etcd image is distroless and may not have `/bin/sh`. If `kubectl exec` fails with "executable not found," install `etcdctl` on a master host (`apt install etcd-client`) and run the commands directly from the master, pointing at the local `2379` socket.

Common causes

- Disk slow (the `scsi1` etcd disk on Proxmox host is congested) — confirm with `iostat -x 1` on the affected master
- DB growth due to a runaway resource (often: Events, or external-secrets ExternalSecret churn, or watch caching gone wrong)
- Network packet loss between members (`etcd_network_peer_round_trip_time_seconds_bucket` shows tail latency)

- Auto-compaction didn't run (check `etcd_debugging_mvcc_db_compaction_keys_total`)

Remediation

For **slow fsync**: - Verify the dedicated etcd disk is healthy: `ssh ubuntu@10.10.120.131 "sudo smartctl -a /dev/sdb"` (Proxmox-backed) - If host disk is shared with noisy neighbors, move the etcd VM disk to a quieter storage pool in Proxmox.

For **growing DB / approaching quota**:

```
# Force compaction
REV=$(kubectl exec $M1 -c etcd -- etcdctl ... endpoint status --write-out json | python3 -c
"import json,sys; print(json.load(sys.stdin)[0]['Status']['header']['revision'])")
kubectl exec $M1 -c etcd -- etcdctl ... compact $REV
# Then defrag on EACH member (one at a time!)
for ip in 10.10.120.131 10.10.120.132 10.10.120.133; do
  kubectl exec $M1 -c etcd -- etcdctl ... --endpoints=https://$ip:2379 defrag
  sleep 30 # let cluster settle
done
```

For **no leader**: - Identify which member is suspect (`endpoint health`) - Restart `rke2-server` on the suspect master (one at a time, never all three):

`ssh ubuntu@<bad-master> "sudo systemctl restart rke2-server"` - Wait 60s, re-check `endpoint health --cluster`

Verification

- All 3 endpoints show `HEALTH: true`
- `etcd_server_has_leader == 1` everywhere
- p99 fsync drops back below 100 ms

Escalation: Etcd losing quorum = cluster API frozen. Page immediately if 2 members are unhealthy.

IR-3 — A node goes NotReady

Symptoms

- `kubectl get nodes` shows `dealing-X-N NotReady`
- Grafana alert: `kube_node_status_condition{condition="Ready",status!="true"} > 0`

Diagnosis

```

NODE=dealing-w-2
kubectl describe node $NODE | sed -n '/Conditions:/,/Events:/p'
# The condition messages are usually self-explanatory: MemoryPressure, DiskPressure,
PIDPressure, NetworkUnavailable

# Is the kubelet running?
ssh ubuntu@10.10.120.135 "sudo systemctl status rke2-agent rke2-server 2>&1 | head -20"

# Is the node reachable from masters?
ssh ubuntu@10.10.120.131 "ping -c 3 10.10.120.135"

# kubelet logs
ssh ubuntu@10.10.120.135 "sudo journalctl -u rke2-agent --since '15 min ago' --no-pager |
tail -100"

# Did the node run out of disk on /?
ssh ubuntu@10.10.120.135 "df -h /"

```

Common causes

- **MemoryPressure** → see IR-4 OOM-related
- **DiskPressure** → root or container-runtime disk filling up; clean up old images: `ssh ubuntu@$NODE "sudo crictl rmi --prune"`
- **NetworkUnavailable** → Cilium agent broken on this node — `kubectl -n kube-system delete pod <cilium-pod-on-this-node>`
- kubelet hung (PLEG slow) → `kubelet_pleg_relist_duration_seconds:p99 > 3s` is the leading indicator; restart rke2-agent

Remediation

```

# Try restarting rke2-agent first (workers) or rke2-server (masters)
ssh ubuntu@$NODE "sudo systemctl restart rke2-agent" # worker
ssh ubuntu@$NODE "sudo systemctl restart rke2-server" # master

# If still NotReady after 2 min: cordon, drain, reboot
kubectl cordon $NODE
kubectl drain $NODE --ignore-daemonsets --delete-emptydir-data --force --timeout=5m
ssh ubuntu@$NODE "sudo reboot"
# Wait for the VM to come back, then:
kubectl uncordon $NODE

```

Verification

- `kubectl get nodes` shows Ready
- Pods that were on the node have been rescheduled successfully

Escalation: If a master goes NotReady, etcd quorum tolerates 1 failure — investigate fast but no immediate page. If 2 masters NotReady → page now.

IR-4 — Pod is CrashLooping / OOMKilled / Pending

Symptoms

- `kubectl -n <ns> get pods` shows `CrashLoopBackOff`, `OOMKilled`, or `Pending`
- Grafana alert: `kube_pod_container_status_restarts_total[15m] > 5` or `container_oom_events_total > 0`

Diagnosis

```

NS=production; POD=myapp-xxx

# What state is the container in?
kubectl -n $NS get pod $POD -o jsonpath='{.status.containerStatuses[*]}' | python3 -m json.tool

# Why was it last terminated?
kubectl -n $NS describe pod $POD | sed -n '/Last State:/,+10p'

# Logs from the previous instance
kubectl -n $NS logs $POD --previous --tail=100

# Was it OOM-killed? Look at container_oom_events_total in Grafana for this pod.
# Or on the host:
ssh ubuntu@<node-where-pod-ran> "sudo dmesg -T | grep -i oom | tail -20"

# For Pending pods – why isn't it scheduling?
kubectl -n $NS describe pod $POD | sed -n '/Events:/,$p'
# Typical messages:
# - "0/6 nodes are available: insufficient cpu, insufficient memory"
# - "node(s) had untolerated taint"
# - "0/6 nodes are available: pod has unbound immediate PersistentVolumeClaims"

```

Common causes & fixes

Symptom	Likely cause	Action
<code>CrashLoopBackOff</code> , exit code 0	App exits cleanly but loops	Bug in app — check logs
<code>CrashLoopBackOff</code> , exit code 1+	App error on start	Check logs, fix config
<code>OOMKilled</code> (137)	Hit memory limit	Raise <code>resources.limits.memory</code> ; profile memory usage
<code>Error: ImagePullBackOff</code>	Bad image / no auth	<code>kubectl describe</code> for exact error; check <code>imagePullSecrets</code>
<code>Pending</code> , "insufficient cpu/memory"	Cluster full	Scale up cluster or right-size the pod's <code>resources.requests</code>
<code>Pending</code> , "untolerated taint"	Node is tainted	Add toleration or untaint node
<code>Pending</code> , "unbound PVC"	StorageClass / PV unavailable	See IR-7 storage

Verification

- `kubectl get pod -w` shows `Running` 1/1 and stable for ≥ 5 minutes

IR-5 — DNS resolution failing in pods

Symptoms

- App can't resolve service names or external hostnames
- `nslookup` from inside a pod returns `Can't find` / NXDOMAIN
- Suspicious metric: `coredns_dns_response_rcode_count_total{rcode=~"SERVFAIL|REFUSED"}`

Diagnosis

```
# 1. Test from inside a debug pod
kubectl run dnstest --image=curlimages/curl:8.10.1 --rm -it --restart=Never -- sh -c '
echo "--- cluster DNS server ---"
cat /etc/resolv.conf
echo "--- in-cluster service ---"
nslookup kubernetes.default.svc.cluster.local
echo "--- external ---"
nslookup mimir.stackflow.org
'

# 2. Are CoreDNS pods healthy?
kubectl -n kube-system get pods -l k8s-app=kube-dns -o wide

# 3. CoreDNS Corefile (custom rules for cluster + stackflow.org)
kubectl -n kube-system get cm rke2-coredns-rke2-coredns -o jsonpath='{.data.Corefile}'

# 4. CoreDNS metrics
kubectl -n kube-system port-forward svc/rke2-coredns-rke2-coredns 9153:9153 &
curl -s localhost:9153/metrics | grep '^coredns_dns_request_count_total' | head
kill %1
```

Common causes

- All CoreDNS pods on one node, node Notready → autoscaler should add replicas; verify
- The Corefile is missing an entry (we hit this with `stackflow.org` — see [the live Corefile](#))
- A pod's `dnsPolicy` is `Default` (uses node's `/etc/resolv.conf`) instead of `ClusterFirst` — won't resolve cluster services
- NetworkPolicy blocks egress to port 53 (see IR-6)

Remediation

- Restart CoreDNS: `kubectl -n kube-system rollout restart deploy/rke2-coredns-rke2-coredns`
- Add missing zone to Corefile via the `rke2-coredns` HelmChartConfig in `rke2/configs/rke2-coredns-config.yaml`, then reapply (helm-controller picks it up)

IR-6 — Pod can't reach another pod / service / external endpoint

Symptoms

- Application timeouts, "connection refused", `context deadline exceeded`
- Grafana alert: `cilium_drop_count_total:rate5m > 100` (tunable)

Diagnosis — *always check Hubble drop verdicts FIRST*

```
# 1. From the source pod's node, run Hubble observe filtered to the source pod
NODE=$(kubectl -n production get pod myapp-xxx -o jsonpath='{.spec.nodeName}')
AGENT=$(kubectl -n kube-system get pod -l k8s-app=cilium --field-selector spec.nodeName=$NODE -o jsonpath='{.items[0].metadata.name}')
SRC_POD_IP=$(kubectl -n production get pod myapp-xxx -o jsonpath='{.status.podIP}')

kubectl -n kube-system exec $AGENT -c cilium-agent -- hubble observe --from-ip $SRC_POD_IP -last 200

# 2. Specifically look at DROPPED with a reason
kubectl -n kube-system exec $AGENT -c cilium-agent -- hubble observe --from-ip $SRC_POD_IP -verdict DROPPED --last 50

# 3. If verdict says "Policy denied" → it's a NetworkPolicy or CCNP. Find which one.
kubectl get netpol,cnp,ccnp -A | grep -i <namespace-or-name>

# 4. Test endpoint reachability from a debug pod on the same node
kubectl run debug --rm -it --image=curlimages/curl:8.10.1 --restart=Never \
  --overrides='{ "spec": { "nodeName": "'$NODE'" } }' \
  -- sh -c "curl -sv --max-time 5 http://<target-service>:<port>/"
```

Common causes

- `Policy denied DROPPED` → NetworkPolicy / CCNP blocks egress. Identify which selector matches the source pod (Cilium endpoint enforcement: `kubectl -n kube-system exec $AGENT -c cilium-agent -- cilium-dbg endpoint list`).
- `no route to host` → target IP not in any cluster identity; for pod → host-IP traffic, you need the `CCNP we added` or equivalent (`toEntities: [host, remote-node]`).
- `Connection refused` → reaches destination but no listener (app crashed / wrong port)
- `Connection timed out` to external → egress firewall, DNS issue, or default-deny netpol

Remediation

- For policy-denied: edit the policy to allow the needed egress (a CCNP for entities, a K8s NetworkPolicy with pod/namespace selectors for in-cluster pods)
- The cluster-wide `allow-pods-to-kube-apiserver` CCNP covers traffic to masters' `kube-apiserver` entity — that's why kubectl-from-pod works without further configuration

Verification

- Repeat the failing curl/test from inside the source pod — should succeed
- Hubble observe shows `to-endpoint FORWARDED`

See also: [memory:check-hubble-drops-first](#) — Hubble drop verdicts are always the first thing to look at when packets vanish.

IR-7 — Certificate renewal failed

Symptoms

- TLS errors at Ingress (browsers warn about expired cert)
- cert-manager event: `Failed to renew Certificate`
- `kubectl get certificate -A` shows `Ready: False`

Diagnosis

```
kubectl get certificate -A
kubectl describe certificate <name> -n <ns> | sed -n '/Status:/,$p'
kubectl get challenge -A
kubectl get order -A

# cert-manager controller logs
kubectl -n cert-manager logs -l app.kubernetes.io/component=controller --tail=200
```

Common causes

- Issuer is not Ready (in this cluster `letsencrypt-prod` and `letsencrypt-staging` are `NotReady` — they need DNS01/HTTP01 solver configuration)
- DNS01 challenge needs DNS provider creds in a Secret
- HTTP01 challenge requires Ingress to be reachable from Let's Encrypt's servers (won't work on `*.internal` hostnames)
- Rate limit hit on Let's Encrypt (use `letsencrypt-staging` while debugging)

Remediation

- For internal hostnames (`*.dealing.internal`, `cluster.internal`): use `cluster-ca-issuer` (the internal CA), not Let's Encrypt
- For public hostnames: fix the solver config in the ClusterIssuer

IR-8 — Prometheus stops writing to Mimir / metrics gap in Grafana

Symptoms

- Grafana dashboards show "No data" for the last X minutes
- Grafana alert: `absent(up{job="kube-prometheus-stack-prometheus"})`

Diagnosis

```
# 1. Is the Prometheus pod healthy?
kubectl -n monitoring get pod -l app.kubernetes.io/name=prometheus

# 2. Is remote-write working?
kubectl -n monitoring exec prometheus-kube-prometheus-stack-prometheus-0 -c prometheus -- \
  wget -qO- 'http://localhost:9090/api/v1/query?
  query=prometheus_remote_storage_samples_pending'

# 3. Recent remote-write errors
kubectl -n monitoring exec prometheus-kube-prometheus-stack-prometheus-0 -c prometheus -- \
  wget -qO- 'http://localhost:9090/api/v1/query?
  query=rate(prometheus_remote_storage_samples_failed_total[5m])'

# 4. Can the pod reach Mimir?
kubectl -n monitoring exec prometheus-kube-prometheus-stack-prometheus-0 -c prometheus -- \
  wget -qO- --timeout=5 http://mimir.stackflow.org/ready

# 5. Prometheus log
kubectl -n monitoring logs prometheus-kube-prometheus-stack-prometheus-0 -c prometheus --
tail=200
```

Common causes

- Mimir endpoint unreachable (DNS, network, Mimir down)
- Prometheus pod OOM-killed (no memory limit means it eats node RAM)
- Disk full (Prometheus WAL fills)
- NetworkPolicy blocking egress to Mimir (see IR-6)

Remediation

- Restart Prometheus: `kubectl -n monitoring rollout restart sts/prometheus-kube-prometheus-stack-prometheus`
- If WAL is corrupted: delete the PVC and let it re-create (loses local 2h cache; remote-written data in Mimir is unaffected)

Common operations

Add a worker node

The cluster uses Terraform to create VMs and `rke2/scripts/install-worker.sh` to join them. Rough procedure:

1. Add a `[workers]` entry in `rke2/configs/inventory.ini` and a worker VM definition in `terraform/proxmox/`
2. `terraform apply` in `terraform/proxmox/` for the dealing cluster
3. `bash rke2/scripts/install-worker.sh dealing-w-N 10.10.120.NN`
4. `kubectl get nodes` should show the new worker within ~2 minutes

Verify: the new worker shows up as `Ready`, runs the cilium DaemonSet pod, gets a pod CIDR (`kube_node_info`).

Remove a worker node

```
NODE=dealing-w-N
kubectl cordon $NODE
kubectl drain $NODE --ignore-daemonsets --delete-emptydir-data --timeout=10m
kubectl delete node $NODE
ssh ubuntu@$NODE "sudo /usr/local/bin/rke2-uninstall.sh"
# Then delete the VM in Proxmox / via terraform.
```

Restart `rke2-server` (the right way)

One master at a time. Two masters down simultaneously breaks etcd quorum.

```
ssh ubuntu@10.10.120.131 "sudo systemctl restart rke2-server"
# Wait until kubectl get nodes shows dealing-m-1 Ready again (~30-60s)
# Then proceed to m-2, then m-3.
```

Restore from an etcd snapshot

Tested: NO. Do this drill before relying on it.

```
# 1. Stop rke2-server on ALL masters
for ip in 10.10.120.131 10.10.120.132 10.10.120.133; do
  ssh ubuntu@$ip "sudo systemctl stop rke2-server"
done

# 2. On the INIT master (m-1), restore from a chosen snapshot
ssh ubuntu@10.10.120.131 "ls -lh /var/lib/rancher/rke2/server/db/snapshots/"
ssh ubuntu@10.10.120.131 "sudo rke2 server --cluster-reset --cluster-reset-restore-path=/
var/lib/rancher/rke2/server/db/snapshots/<chosen-snapshot>"

# 3. On the other masters: remove the etcd db dir and rejoin
for ip in 10.10.120.132 10.10.120.133; do
  ssh ubuntu@$ip "sudo rm -rf /var/lib/rancher/rke2/server/db/etcd/"
  ssh ubuntu@$ip "sudo systemctl start rke2-server"
done
```

Apply a CiliumNetworkPolicy

```
# Test the policy against an existing source pod
kubectl apply -f mypolicy.yaml
kubectl get cnp -n <ns> mypolicy -o jsonpath='{.status.conditions[*]}' | python3 -m json.tool

# Watch what happens to traffic
kubectl -n kube-system exec <agent-on-relevant-node> -c cilium-agent -- \
  hubble observe --from-namespace <source-ns> --last 100
```

Defrag etcd (highly recommended — schedule weekly)

```
# One member at a time, with a 30s pause between
M1=$(kubectl -n kube-system get pod -l component=etcd --field-selector spec.nodeName=dealing -m-1 -o name)
for ip in 10.10.120.131 10.10.120.132 10.10.120.133; do
  kubectl -n kube-system exec $M1 -c etcd -- \
    etcdctl --endpoints=https://$ip:2379 \
    --cacert=/var/lib/rancher/rke2/server/tls/etcd/server-ca.crt \
    --cert=/var/lib/rancher/rke2/server/tls/etcd/server-client.crt \
    --key=/var/lib/rancher/rke2/server/tls/etcd/server-client.key \
    defrag
  sleep 30
done
```

TODO: wrap this in a CronJob; mentioned in the catalog as a pre-handover item.

Helm chart upgrade

Most Helm releases are managed by Terraform. To upgrade:

1. Pin the new version in `terraform/observability/modules/prometheus/main.tf` (or the relevant module)
2. `terraform plan` and review the diff
3. `terraform apply`
4. Watch the rollout: `kubectl -n monitoring rollout status sts/prometheus-...`

For RKE2-bundled charts (Cilium, CoreDNS, metrics-server, snapshot-controller): edit the corresponding

`HelmChartConfig` manifest in `/var/lib/rancher/rke2/server/manifests/` on m-1 (and commit to repo at `rke2/configs/`). RKE2's helm-controller picks it up and reconciles.

Cheat sheet

kubectl one-liners

```
# Sorted by restart count – find your flaky pods
kubectl get pods -A --sort-by='.status.containerStatuses[0].restartCount' | tail -20

# Find pods consuming the most memory right now (needs metrics-server)
kubectl top pods -A --sort-by=memory | head -20

# All workloads in a namespace + their replica health
kubectl get deploy,sts,ds -n <ns> -o wide

# Find which node a pod runs on, fast
kubectl get pod -A -o wide | grep <pod-name>

# Show events sorted by time
kubectl get events -A --sort-by='.lastTimestamp'

# Decode a Secret quickly
kubectl -n <ns> get secret <name> -o jsonpath='{.data}' | python3 -c "import
sys,json,base64;[print(f'{k}: {base64.b64decode(v).decode()}') for k,v in
json.load(sys.stdin).items()]"

# What's still on a node before draining?
kubectl get pods -A -o wide --field-selector=spec.nodeName=<node-name>

# Restart a Deployment without changing anything
kubectl -n <ns> rollout restart deploy/<name>
```

Cilium / Hubble one-liners

```
AGENT=$(kubectl -n kube-system get pod -l k8s-app=cilium --field-selector
spec.nodeName=<node> -o jsonpath='{.items[0].metadata.name}')

# Status overview from one agent
kubectl -n kube-system exec $AGENT -c cilium-agent -- cilium-dbg status --verbose

# Show local endpoints (one row per pod on this node)
kubectl -n kube-system exec $AGENT -c cilium-agent -- cilium-dbg endpoint list

# Show ipcache (which IPs Cilium knows about)
kubectl -n kube-system exec $AGENT -c cilium-agent -- cilium-dbg bpf ipcache list

# Show the Cilium-managed conntrack table
kubectl -n kube-system exec $AGENT -c cilium-agent -- cilium-dbg bpf ct list global

# Hubble – recent flows for a namespace
kubectl -n kube-system exec $AGENT -c cilium-agent -- hubble observe --namespace <ns> --
last 50

# Hubble – recent drops
kubectl -n kube-system exec $AGENT -c cilium-agent -- hubble observe --verdict DROPPED --
last 50

# Hubble – what's a specific pod talking to?
kubectl -n kube-system exec $AGENT -c cilium-agent -- hubble observe --from-pod <ns>/<pod> -
-last 100
```


Node-level commands (SSH first)

```
# RKE2 service status
sudo systemctl status rke2-server      # masters
sudo systemctl status rke2-agent       # workers

# Static-pod manifests (what RKE2 will run)
ls /var/lib/rancher/rke2/agent/pod-manifests/  # workers
ls /var/lib/rancher/rke2/server/manifests/     # masters (HelmChartConfigs + static pods)

# RKE2 logs
sudo journalctl -u rke2-server -f
sudo journalctl -u rke2-agent -f

# Alloy (host metrics + logs)
sudo systemctl status alloy
sudo journalctl -u alloy --since '10 min ago' | tail

# Disk / filesystem
df -h
sudo du -sh /var/lib/rancher/rke2/server/db/* 2>/dev/null # etcd disk usage
sudo du -sh /var/log/pods/                     # pod log directory

# Container runtime
sudo crictl ps                                # what's actually running
sudo crictl images                            # local image cache
sudo crictl rmi --prune                       # garbage-collect unused images
```

Escalation matrix (template — TEAM TO FILL IN)

Severity	When to use	Notify how	Who
P1 (page)	Cluster API down >5 min, etcd quorum lost, all nodes NotReady, ingress LB unreachable, security incident	Teams + phone	<i>on-call human</i>
P2 (notify)	Single node NotReady >15 min, persistent OOM in production, etcd fsync p99 sustained >100 ms, PVC stuck Pending in production	Teams channel	<i>on-call human</i>
P3 (notify quietly)	Single pod restarts, occasional high CPU throttling, Hubble drop rate elevated, non-prod issues	Teams channel	<i>no immediate action</i>
Info	Maintenance / planned change notifications	Teams channel	<i>all engineering</i>

References

- [Workload catalog](#) — what's deployed
- [Cluster handbook](#) — what every component does + port inventory

- [Monitoring playbook](#) — how to read the dashboards day-to-day, panel thresholds, incident-to-RCA workflow
- [Policy reviewer checklist](#) — before adding NetworkPolicy/CNP/CCNP/Kyverno
- [Multi-cluster operations runbook](#) — fleet-level operations across all clusters
- [Generic troubleshooting](#) — generic landing page; points back to this runbook
- Repo: `clusters/dealing/` — kubeconfig, tfvars, terraform state
- Repo: `rke2/configs/` — RKE2 server/worker configs (gitignored, regenerated by terraform per cluster)
- Repo: `security/network-policies/` — cluster NetworkPolicy + CCNP definitions
- Repo: `terraform/observability/` — kube-prometheus-stack values + alert rules

Last updated: [auto] / Add yourself to the changelog when you edit:

Date	Author	What changed
2026-05-21	initial draft	First commit